SMARTY – AI VOICE ASSISTANT WITH AUTOMATION AND TALKBACK RESPONSE SYSTEM

A PROJECT REPORT

Submitted by

Mr. MOHAMMED ARMAAN BAKSHI I

[Register No: 2322J1459]

Under the guidance of

Dr. MUTHUKUMAR P MCA., M.Phil., B.Ed., Ph.D., SET.,

[Head, Department of Computer Applications & Department of Computer Science with Cyber Security]

In partial fulfilment for the award of the degree

Of

BACHELOR OF COMPUTER APPLICATIONS

In

DEPARTMENT OF COMPUTER APPLICATIONS

NILGIRI COLLEGE OF ARTS AND SCIENCE

(AUTONOMOUS)

MARCH 2026

NILGIRI COLLEGE OF ARTS AND SCIENCE

(AUTONOMOUS)

DEPARTMENT OF COMPUTER APPLICATIONS

PROJECT WORK

MARCH - 2026

This is to certify that the project entitled

SMARTY – AI VOICE ASSISTANT WITH AUTOMATION AND TALKBACK RESPONSE SYSTEM

Is a Bonafide record of project work done by

Mr. MOHAMMED ARMAAN BAKSHI I

[Register No: 2322J1459]

Project Submitted in partial fulfilled of the requirement for the award of

Bachelor of Computer Applications during the year 2023-2026

Project Guide **Head of the Department**

Submitted for the project viva-voce examination held on.....

Internal Examiner External Examiner

DECLARATION

I hereby declare that the project work, " **Smarty – AI Voice Assistant with Automation and Talkback response system** " submitted to Nilgiri College of Arts & Science (Autonomous), in partial fulfilment of the requirements for the award of degree **Bachelor of Computer Applications**, is a record of original project work done by me ,during the period of December 2025 to March 2026 under the guidance of **Dr. MUTHUKUMAR P MCA., M.Phil., B.Ed., Ph.D., SET., Head, Department of Computer Applications & Department of Computer Science with Cyber Security,** Nilgiri College of Arts and Science(Autonomous), Thaloor.

Mr. MOHAMMED ARMAAN BAKSHI I

Signature of the Student

PLACE :

DATE :

ACKNOWLEDGEMENT

I hereby express my sincere gratitude to the people, whose cooperation has helped me for the successful completion of my project work. I would like to thank them deep from my heart, for the valuable assistance they rendered for me.

I express my deep sense of gratitude to our honorable Principal **Dr. P BALA SHANMUGA DEVI, MA., M.Phil., Ph.D., SET., Principal,** Nilgiri College of Arts and Science (Autonomous), for the outstanding facilities provided to carry out my project work.

I am extremely grateful display indebted to our **Dr. MUTHUKUMAR P MCA., M.Phil., B.Ed., Ph.D., SET., Head, Department of Computer Applications & Department of Computer Science with Cyber Security,** Nilgiri College of Arts and Science (Autonomous)Thaloor, for the valuable facilities and help provided to me.

I express my deep sense of esteem to my guide **Dr. MUTHUKUMAR P MCA., M.Phil., B.Ed., Ph.D., SET., Head, Department of Computer Applications & Department of Computer Science with Cyber Security,** Nilgiri College of Arts and Science (Autonomous) Thaloor**,** for all his encouragement, valuable advice and timely instructions at various stages of my project work.

I am thankfully recollecting the helping mentality and kind cooperation rendered by my intimate friends, family and all my dear and near ones for the successful completion of my project work.

MOHAMMED ARMAAN BAKSHI I

LIST OF FIGURES

Fig 1	:	Overall System Architecture of Smarty Voice Assistant
Fig 2	:	Voice Command Processing Workflow
Fig 3	:	Speech Recognition Process
Fig 4	:	Local LLM Processing Using Ollama
Fig 5	:	User Interaction with Voice Assistant
Fig 6	:	Sample Talkback Response Checking for Smarty AI Voice Assistant
Fig 7	:	To Open Applications using Voice Commands
Fig 8	:	Checking Auto Typing functions using Voice in Google Browser
Fig 9	:	To Play Media using Voice Commands
Fig 10	:	Playing Video on Command in YouTube Web Browser
Fig 11	:	Displays WhatsApp Web on Command
Fig 12	:	Displaying Today's News on Command
Fig 13	:	Auto Typing in Google Browser

LIST OF ABBREVIATIONS

AI	**:**	Artificial Intelligence
LLM	**:**	Large Language Model
ASR	**:**	Automatic Speech Recognition
STT	**:**	Speech-to-Text
TTS	**:**	Text-to-Speech
HCI	**:**	Human-Computer Interaction
API	**:**	Application Programming Interface
OS	**:**	Operating System
CPU	**:**	Central Processing Unit
RAM	**:**	Random Access Memory
GPU	**:**	Graphics Processing Unit
HTTP	**:**	Hypertext Transfer Protocol
NLP	**:**	Natural Language Processing

ABSTRACT

This project presents **SMARTY – AI Voice Assistant with Automation and Talkback Response System**, a Python-based intelligent assistant designed to perform system automation and conversational tasks through voice interaction. The system captures voice commands using a microphone and converts them into text through the Speech Recognition library. The processed command is then interpreted using a locally deployed Large Language Model integrated through the Ollama platform, enabling the assistant to understand user queries and generate meaningful responses.

The assistant can execute several automation tasks such as opening applications, searching the web, retrieving system information, and performing basic computer operations. To enhance user interaction, the system includes a talkback response mechanism using the pyttsx3 text-to-speech engine, which allows the assistant to communicate responses audibly.

Unlike traditional cloud-based voice assistants, the proposed system performs most of its processing locally, which improves response speed and enhances user privacy. The modular architecture of the system allows easy expansion and integration with additional features in the future.

This project demonstrates how artificial intelligence, speech processing, and automation technologies can be combined to develop a practical and efficient voice-controlled computing system that improves accessibility and user experience.

TABLE OF CONTENTS

S.NO	TITLE	PAGE No
1.	**PROBLEM DEFINITION**	01
	1.1 Overview	01
	1.2 Problem Statement	01
2.	**INTRODUCTION**	02
	2.1 System Specification	03
	2.1.1 Hardware Specification	03
	2.1.2 Software Specification	03
3.	**SYSTEM STUDY**	04
	3.1 Existing System	04
	3.1.1 Drawbacks	04
	3.2 Proposed System	05
	3.2.1 Features	05
	PACKAGE SELECTION	06
4.	**LITERATURE SURVEY**	08
5.	**SYSTEM DESIGN AND DEVELOPMENT**	09
	5.1 Input Design	09
	5.2 Output Design	10
	5.3 Description of Modules	12
6.	**TESTING AND IMPLEMENTATION**	13
	Command Test Table	15
	Implementation	16
7.	**CONCLUSION**	17
8.	**SCOPE OF FUTURE ENHANCEMENT**	18
9.	**REFERENCES**	19
10.	**APPENDICES**	20
	A. DIAGRAMS	20
	B. SAMPLE CODING	22
	C. SAMPLE INPUT	26
	D. SAMPLE OUTPUT	27

PROBLEM DEFINITION

1. PROBLEM DEFINITION

1.1 OVERVIEW

Due to the rapid growth of Artificial Intelligence, the concept of voice-based interaction has become an integral part of the current computing environment. The conventional method of interacting with a computer involves the use of a keyboard and a graphical user interface. This method is not always convenient for the user. Voice-based interaction is a much more convenient method of interacting with a computer. The user can perform various operations using a speech-based interface.

The current voice-based interfaces, such as Alexa, Siri, Google Assistant, etc., depend on the concept of cloud computing to perform the required operations. This has raised several concerns related to the internet connection and the response time of the system. The current voice-based interfaces do not function effectively in an environment where the internet connection is low.

The proposed Smarty AI Voice Assistant is a system that addresses the current challenges related to the conventional voice-based interfaces. The proposed system is a combination of speech recognition, AI, and automation. The proposed system is able to recognize the speech of the user, perform the required automation tasks, and respond to the user using a talkback system.

1.2 PROBLEM STATEMENT

The major challenge in modern voice assistant systems is the dependency on cloud-based processing and limited customization capabilities.

Existing systems face the following issues:

- Dependence on continuous internet connectivity
- Privacy risks due to cloud data transmission
- Limited control over system automation
- Higher latency due to remote processing.

The project will utilize the ability to process voice commands locally using its speech recognition and language model deployed locally using Ollama, which will provide a quick response and efficient automation capabilities.

INTRODUCTION

2. INTRODUCTION

Voice assistants have become an integral part of the current computing environment. These tools enable users to interact with the computer using natural language.

Smarty AI Voice Assistant is a tool designed to work as an intelligent assistant for the user. It is able to perform tasks using voice commands. The tool works by receiving the speech of the user through a microphone, converting the speech to text using speech recognition, using a local AI model to process the text, and then converting the response to speech using text-to-speech.

This tool uses several technologies to enable the user to interact seamlessly. It uses speech recognition, a language model, and automation tools. The tool uses a local language model through the Ollama tool. This tool allows the user to interact without the need for a cloud-based model. It allows the user to customize the automation tool according to their requirements.

2.1 SYSTEM SPECIFICATION

2.1.1 Hardware Requirement:

Processor	:	AMD ryzen 5 or Intel i5
RAM	:	8 GB
ROM	:	512 GB
MICROPHONE	:	Standard Default Microphone
SPEAKER	:	Default Audio output

2.1.2 Software Requirement:

Front End : Python (Terminal)

Backend : Python Libraries

- speech_recognition
- pyttsx3
- pyautogui
- psutil
- requests
- pywhatkit

OS : Windows 10 or higher

LLM : Ollama (gemma3:1b)

IDE : Visual Studio Code

SYSTEM STUDY

3. SYSTEM STUDY

3.1 EXISTING SYSTEM

Existing voice assistant solutions such as Amazon Alexa, Google Assistant, Microsoft Cortana, and Apple Siri are primarily designed as cloud-based systems. These assistants send voice recordings to remote servers for processing, receive generated responses, and return them to the user. While feature-rich, these systems share several fundamental limitations when it comes to privacy-sensitive or offline use cases.

3.1.1 Drawbacks:

- Internet Dependency
- Privacy Risks
- Limited System Automation
- High Latency
- Customization Constraints
- Hardware Dependency

3.2 PROPOSED SYSTEM

Smarty is a locally deployed Python-based voice assistant that eliminates cloud dependency by running its AI reasoning engine directly on the user's machine using Ollama. It combines Google Speech Recognition for accurate voice-to-text conversion with a locally hosted gemma3:1b LLM for general conversation and a rich automation layer using pyautogui and psutil.

3.2.1 Features

- Offline AI Reasoning
- Voice-Activated Control
- Talkback Response System
- Application & Web Automation
- Media Playback
- System Volume Control
- System Status Monitoring
- Keyboard Automation
- Date & Time Queries
- Conversational Memory
- WhatsApp Integration
- News Access

PACKAGE SELECTION

speech_recognition

Used to capture audio from the microphone and convert spoken voice commands into text using Google's Speech Recognition API. Handles ambient noise calibration, listen timeouts, and recognition exceptions.

pyttsx3

Provides offline Text-to-Speech synthesis using the Windows SAPI5 engine. Configured with a female voice at 170 WPM to deliver Smarty's talkback responses audibly without internet dependency.

pyautogui

Handles GUI and keyboard automation — simulating keypresses (Win key, Alt+F4, volume keys), typing text, and minimizing windows. Enables Smarty to control the desktop programmatically.

psutil

Used for system process management and performance monitoring. Powers the "close app by name" feature by iterating running processes and reports real-time CPU and RAM usage on demand.

requests

Sends HTTP POST requests to the locally running Ollama API endpoint (http://localhost:11434/api/generate) to query the gemma3:1b language model for conversational AI responses.

pywhatkit

Handles YouTube video playback by searching and auto-playing videos in the browser via pywhatkit.playonyt().

webbrowser

A Python standard library module used to open URLs in the default browser — powering commands for YouTube, Google, Instagram, WhatsApp Web, Spotify, and Google News.

threading

Used to run TTS speech synthesis in a separate daemon thread, preventing the voice output from blocking the main program loop while still completing before the next action.

datetime

Standard library module used to fetch and format the current time and date for Smarty's time/date query responses.

time

Standard library module used for startup delay and brief pauses between automation actions (e.g., waiting for the Start Menu to open before typing an app name).

LITERATURE SURVEY

4. LITERATURE SURVEY

The evolution of voice assistant and conversational AI technologies has witnessed significant advancements in the last decade. Traditionally, these technologies used rule-based approaches and relied on matching keywords and simple intent detection mechanisms.

However, the emergence of deep neural networks and transformer-based language models has revolutionized the field of natural language processing. GPT, BERT, and Gemma are such language models that have greatly impacted the development of natural language understanding and conversation. The Gemma model family, developed by Google DeepMind, also offers a lightweight version known as gemma3:1b, which can be executed on a personal machine, making the concept of local AI deployment a reality.

Speech Recognition technologies have also witnessed significant advancements in the last decade due to the emergence of deep learning technologies. Google's Web Speech API and CMU Sphinx are two popular technologies used in the development of Speech Recognition technologies. The SpeechRecognition library in Python offers a simple abstraction over these technologies, and Google's Cloud API offers the best accuracy in real-time Speech Recognition at the cost of internet connectivity.

Text-to-Speech synthesis has also changed from robotic-sounding FESTIVAL/espeak engines to more modern Neural TTS systems like Tacotron, WaveNet, etc. The pyttsx3 library offers an offline solution for TTS synthesis by leveraging OS native SAPI5 for Windows and espeak for Linux engines.

These tools have collectively contributed to building Smarty, which is an integration of speech recognition, offline LLM inference, and cross-application desktop automation tools.

SYSTEM DESIGN AND DEVELOPMENT

5. SYSTEM DESIGN AND DEVELOPMENT

The design of the Smarty system is based on a modular pipeline architecture. The voice input is received, transcribed, processed by a command parser, routed to the appropriate handler module, and spoken out loud. For unrecognized commands, it is routed to the local LLM for intelligent response generation.

5.1 INPUT DESIGN

The primary input to Smarty is speech captured via the system microphone. The SpeechRecognition library continuously listens using PyAudio/ALSA. It performs ambient noise calibration (0.5s duration) before each listen cycle, using a 5-second timeout and an 8-second phrase time limit.

Once audio is captured, it is sent to Google's Speech Recognition API for transcription into lowercase text. This text string is then passed into the command routing logic of the main loop. If the transcription fails or is empty, the loop continues and listens again. The input design ensures robustness through exception handling for WaitTimeoutError and generic errors.

Secondary input includes the user's system environment: the microphone hardware, ambient noise level, and speech clarity all influence transcription quality. No graphical interface is required; the terminal serves as the visual log for both recognized commands and system responses.

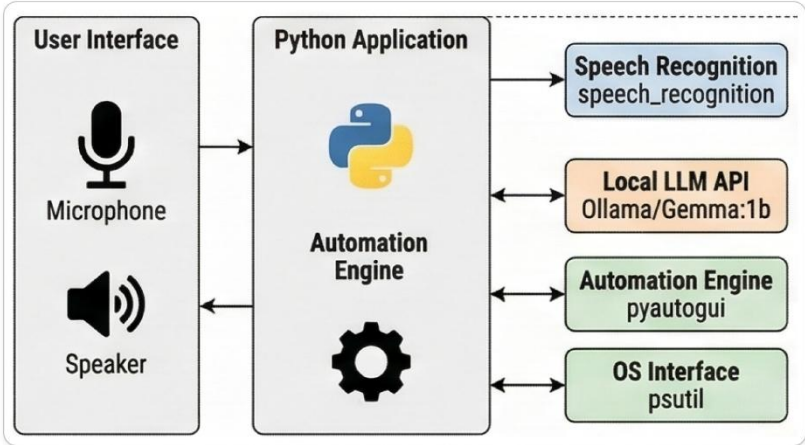


Fig 1: Overall System Architecture of Smarty Voice Assistant.
Shows the overall structure of the Smarty voice assistant and how different modules interact.

5.2 OUTPUT DESIGN

Smarty produces two forms of output for every interaction:

- Spoken Response (Talkback): The pyttsx3 engine is initialized with the SAPI5 backend, configured with a female voice (voices[1]) at a speech rate of 170 words per minute. The speak() function runs synthesis in a background daemon thread and waits for completion before the next action, ensuring responses are delivered sequentially.
- Terminal Log: Every recognized user command and Smarty response is printed to the console for session monitoring and debugging.

Automation outputs include browser windows being opened, applications being launched via the Windows Start Menu (pyautogui), keystrokes being typed, and system-level events such as volume changes. The output design prioritizes voice feedback for every user action so the interaction feels natural and eyes-free.

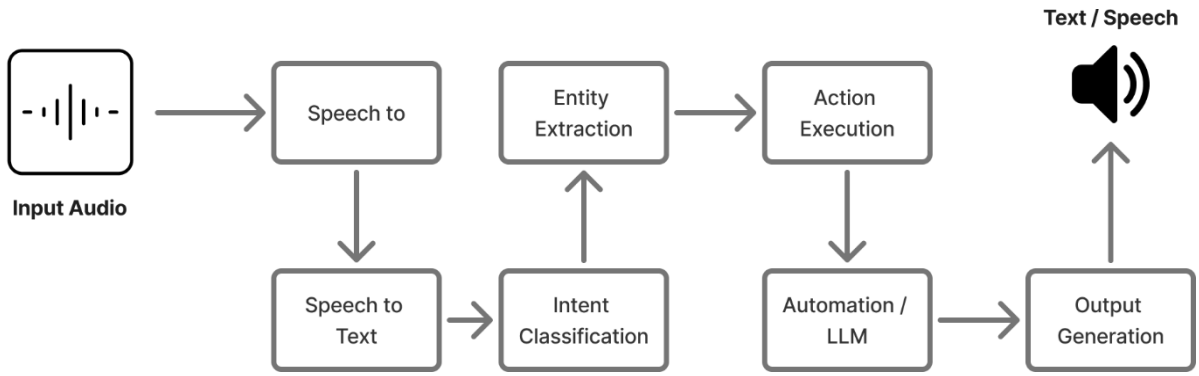


Fig 2: Voice Command Processing Workflow.

Illustrates the step-by-step process of converting voice input into system actions.

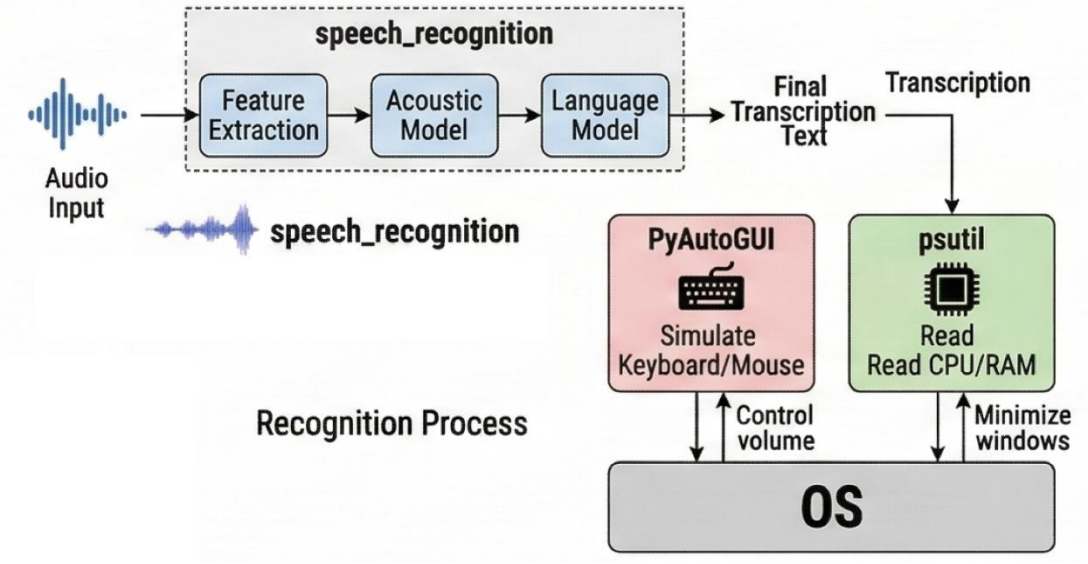


Fig 3: Speech Recognition Process.

Demonstrates how spoken commands are converted into text using speech recognition.

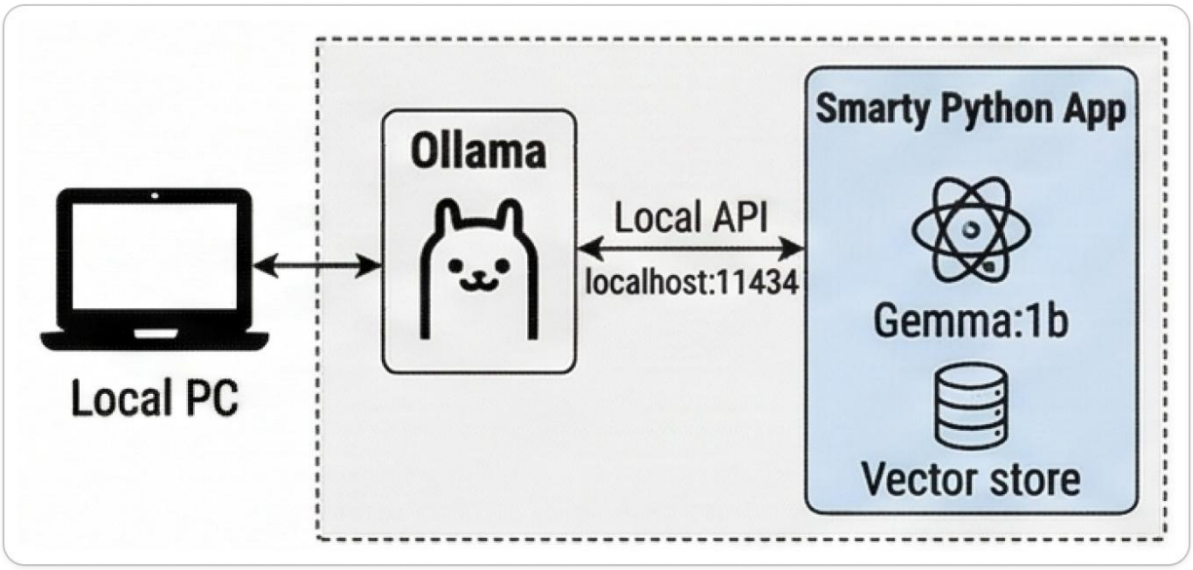


Fig 4: Local LLM Processing using Ollama.

Shows how the local language model processes user commands and generates responses.

5.3 DECRIPTION OF MODULES

Module 1: Speech Recognition (listen)

Captures microphone audio and converts it to lowercase text using the Google Speech Recognition API. Handles ambient noise calibration, listen timeout, and recognition errors. Returns an empty string on failure to allow the loop to continue gracefully.

Module 2: Text-to-Speech Talkback (speak / _speak_thread)

Synthesizes spoken responses using pyttsx3 SAPI5. Uses threading to run speech in a separate daemon thread while blocking the main thread until speech completes. Supports a pause_requested flag to suppress speech when required.

Module 3: AI Thinking Engine (think)

Sends the user's input to the locally running Ollama API (gemma3:1b model) via HTTP POST. Constructs a structured prompt that instructs the model to respond as Smarty with brief, emoji-free conversational answers. Handles connection errors gracefully and returns a fallback message if Ollama is unavailable.

Module 4: Application & Web Automation

Opens websites (YouTube, Google, Instagram, WhatsApp, Spotify, Google News) using the webbrowser module. Launches local Windows applications by simulating Win key press and typing via pyautogui. Closes applications by process name using psutil.process_iter.

Module 5: Media Playback

Plays YouTube videos by extracting the query from the voice command and invoking pywhatkit.playonyt(). Handles Spotify by opening a search URL in the browser.

Module 6: System Automation

Handles keyboard shortcuts using pyautogui.hotkey (Win+D to minimize, Alt+F4 to close windows). Controls volume using pyautogui.press with volumeup, volumedown, and volumemute keys. Reports CPU and RAM usage using psutil.cpu_percent() and psutil.virtual_memory().

Module 7: Conversation Memory

Maintains a conversation_history list that stores the last 6 user and Smarty exchanges. This context is prepended to each AI prompt to enable context-aware multi-turn conversation.

TESTING AND IMPLEMENTATION

6. TESTING AND IMPLEMENTATION

Testing of Smarty was conducted across multiple phases to ensure that all modules function correctly both independently and in combination. The voice-driven nature of the system required testing across varied environments, voice styles, and command patterns.

TESTING OBJECTIVE

- Verify that speech recognition correctly transcribes a wide range of voice commands.
- Confirm that command routing dispatches correctly to each handler module.
- Validate that automation tasks (app opening, typing, volume control) execute reliably.
- Ensure the Ollama LLM responds appropriately to conversational queries.
- Test graceful error handling for microphone unavailability and Ollama connection failures.

TESTING STRATEGIES

- Unit Testing
- Integration Testing
- System Testing
- Performance Testing

UNIT TESTING

Each module was tested independently. The speak() function was tested with various string inputs to confirm audio output. The listen() function was tested with different ambient noise levels and silence timeouts. The think() function was tested with Ollama running and deliberately stopped to verify fallback behavior. The close_app() function was tested against running and non-running process names.

INTEGRATION TESTING

Integration testing verified the pipeline from voice input to spoken output. Commands were spoken and the full cycle was observed: capture → transcribe → route → execute → respond. Multi-turn conversational exchanges were tested to verify that conversation_history correctly influenced subsequent AI responses.

SYSTEM TESTING

The complete assistant was run in a real desktop environment. All command categories (greetings, time/date, web, apps, media, volume, system status, typing, AI chat) were tested end-to-end. Edge cases including ambiguous commands, background noise, and partial matches were also evaluated.

PERFORMANCE TESTING

Response latency was measured for each module. Voice transcription (Google API) averaged 1–2 seconds over typical network conditions. Ollama inference for gemma3:1b averaged 2–4 seconds on an Intel i5 system. Automation tasks (app open, volume change) were near instant. TTS response time was proportional to response length at 170 WPM.

COMMAND TEST TABLE

Voice Command / Trigger	Action Performed
hi / hello / hey smarty	Responds with a greeting
what time is it	Speaks the current time (12-hour format)
what is today's date	Speaks the current date
open youtube	Opens YouTube in the default browser
open google	Opens Google in the default browser
open whatsapp	Opens WhatsApp Web
open <app name>	Searches and opens the application via Windows Start
close window / close browser	Sends Alt+F4 to close the active window
close <app name>	Kills the named process using psutil
minimize / hide	Minimizes all windows with Win+D
volume up	Presses volume up key 5 times
volume down	Presses volume down key 5 times
mute	Toggles system mute
type / write <text>	Types the specified text using pyautogui
play video <query>	Plays a YouTube video matching the query
play song <query>	Opens Spotify search for the query
system status / performance	Speaks current CPU and RAM usage
news	Opens Google News in the browser
call <name>	Opens a WhatsApp Web contact session
can you hear me	Responds confirming audio is clear
exit / stop / shutdown / bye	Speaks goodbye and terminates the program

IMPLEMENTATION

Smarty is implemented as a single Python script that runs in an infinite loop on any Windows 10+ machine with a microphone. Installation requires Python 3.x with the following libraries installed via pip: speech_recognition, pyttsx3, pyautogui, psutil, requests, pywhatkit. The Ollama platform must be installed and running locally with the gemma3:1b model pulled before starting Smarty.

On startup, Smarty sleeps for one second to allow audio hardware to initialize, then speaks its startup greeting: "Smarty AI system online, Hello Sir!". The main loop then continuously listens for commands, routes them through the command handlers, executes the appropriate action, and returns voice feedback. The loop only exits when the user speaks an exit command.

Fig 5 : User Interaction with Voice Assistant.

Represents how the assistant executes system tasks like opening applications or websites.

CONCLUSION

7. CONCLUSION

This project was a complete success in demonstrating the design and implementation of the Smarty; Locally Deployed AI Voice Assistant with Automation and Talkback Response System. The project was successful in its purpose of creating a locally deployed AI voice assistant with automation and a talkback response system without the use of any cloud-based AI services.

Using the Google Speech Recognition API for voice transcription, the Ollama gemma3:1b local AI model, the pyttsx3 API for natural voice responses, and the pyautogui/psutil API for automation, the project was a complete success in demonstrating the use of open-source tools for the development of a voice assistant.

The modular design of the project allows for the easy integration of new command handlers, the use of a different local LLM, and the automation features. The project was also successful in demonstrating the functionality of all the command categories, with the correct fall-back behavior for cases when the microphone or Ollama services are not available.

This project provides a solid foundation for the development of locally deployed AI voice assistants for personal and professional use.

SCOPE OF FUTURE ENCHANCEMENT

8. SCOPE OF FUTURE ENHANCEMENT

Wake Word Detection: Implement a passive wake word listener (e.g., "Hey Smarty") using Porcupine or Vosk so the assistant activates without requiring the user to manually run the script. GUI Dashboard: Build a PyQt5 or Tkinter interface displaying recognized commands, AI responses, system stats, and a conversation log in real time. Upgraded Local LLM: Replace gemma3:1b with a larger Ollama model (e.g., Llama 3, Mistral) for richer and more accurate AI responses. WhatsApp Messaging: Integrate pywhatkit.sendwhatmsg() to send WhatsApp messages by voice. Email Integration: Add Gmail or SMTP-based email composition and sending via voice command. Calendar & Reminders: Connect to a local or Google Calendar API to set reminders and read upcoming events. Custom Command Scripting: Allow users to define custom voice command, action mappings via a configuration file. Multi-Language Support: Add support for regional languages in both speech recognition and TTS output. Portable Packaging: Package Smarty as a standalone Windows executable using PyInstaller for easy distribution. File System Automation: Add voice commands to search, open, move, or rename files using os and shutil.

REFERENCES

9. REFERENCES

Book Reference

- Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing (3rd ed.). Pearson.
- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

Web Reference

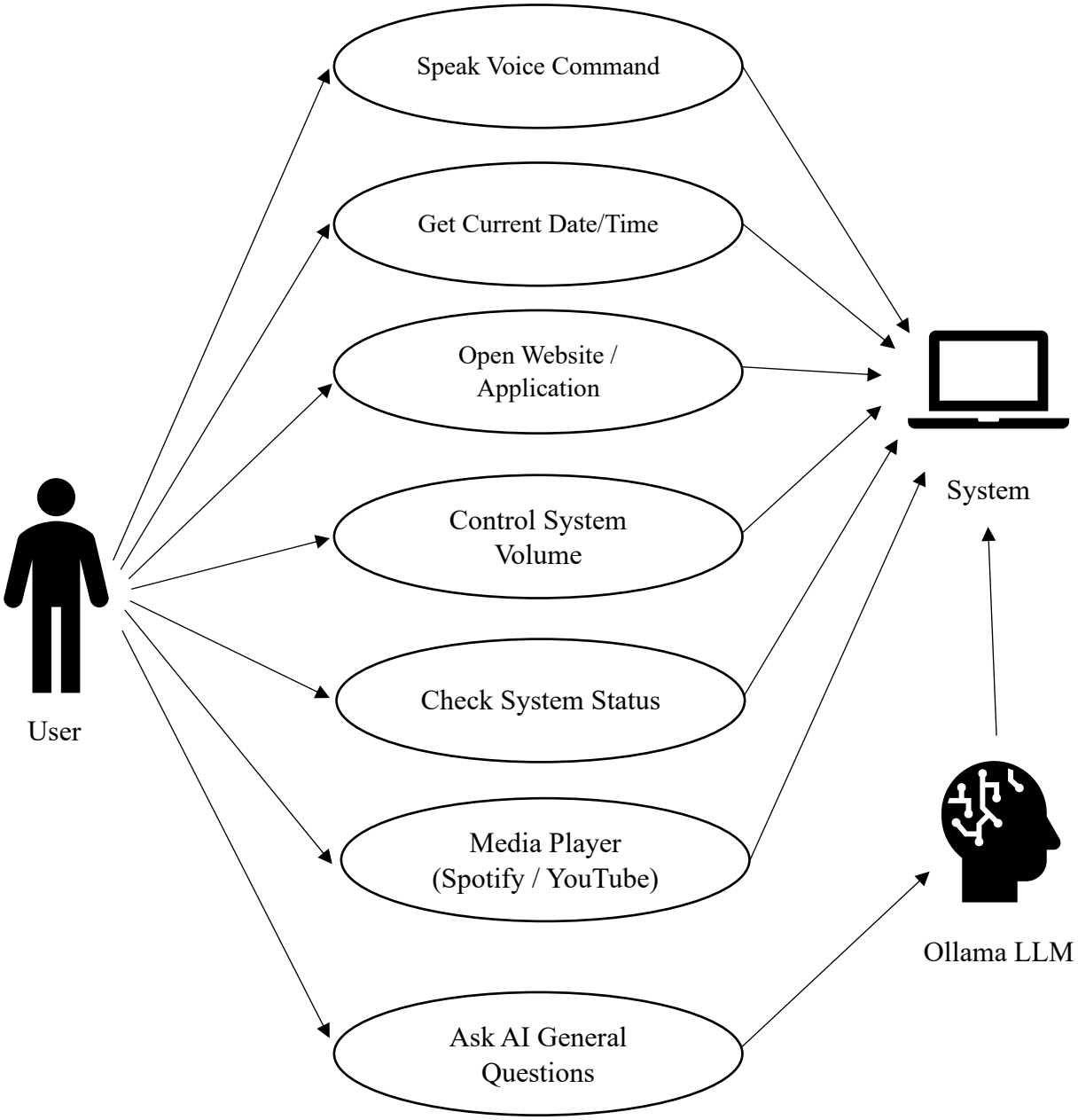
- https://pypi.org/project/SpeechRecognition/
- https://pypi.org/project/pyttsx3/
- https://ollama.com
- https://ai.google.dev/gemma
- https://pyautogui.readthedocs.io
- https://psutil.readthedocs.io
- https://pypi.org/project/pywhatkit/

APPENDICES

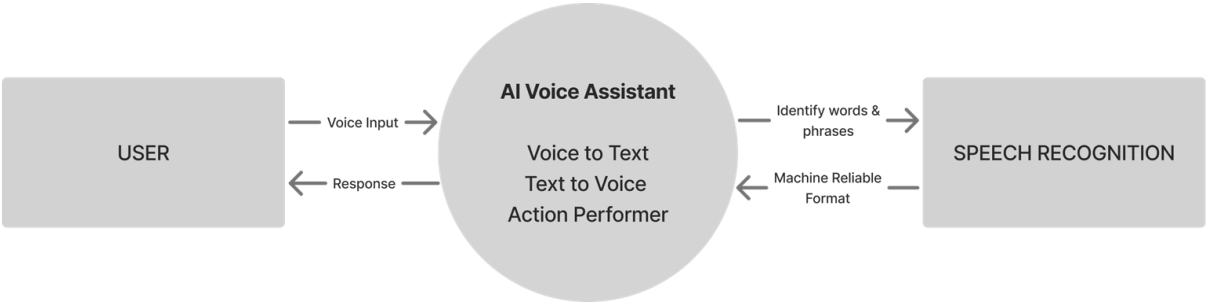
10. APPENDICES

A. DIAGRAM

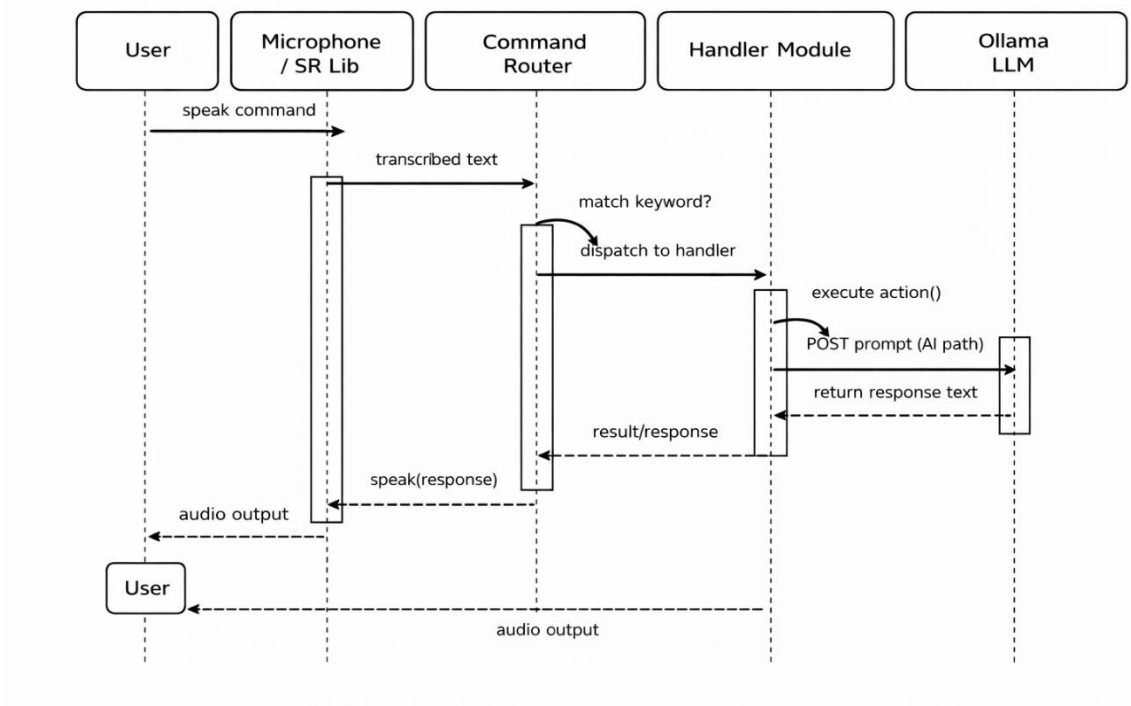
1. USE CASE DIAGRAM



2. DFD DIAGRAM



3. SEQUENCE DIAGRAM



B. SAMPLE CODE

```
import time
import webbrowser
import threading
import speech_recognition as sr
import pyttsx3
import pyautogui
import psutil
import requests
from datetime import datetime
import pywhatkit

#Local Model
OLLAMA_URL = "http://localhost:11434/api/generate"
OLLAMA_MODEL = "gemma3:1b"
conversation_history = []
is_speaking = False
pause_requested = False

#Speak Method
def speak(text):
    global is_speaking, pause_requested
    if pause_requested:
        return
    print("Smarty:", text)
    is_speaking = True
    t = threading.Thread(target=_speak_thread, args=(text,))
    t.daemon = True
    t.start()
    t.join()  # Wait for speech to finish before continuing
    is_speaking = False
```

```
def _speak_thread(text):
    global pause_requested
    engine = pyttsx3.init("sapi5")
    voices = engine.getProperty("voices")
    engine.setProperty("voice", voices[1].id)
    engine.setProperty("rate", 170)
    engine.say(text)
    engine.runAndWait()
    engine.stop()

#Listening Method
def listen():
    global pause_requested, is_speaking
    r = sr.Recognizer()
    with sr.Microphone() as source:
        print("Listening...")
        r.adjust_for_ambient_noise(source, duration=0.5)
        try:
            audio = r.listen(source, timeout=5, phrase_time_limit=8)
        except sr.WaitTimeoutError:
            return ""
    try:
        command = r.recognize_google(audio)
        print("You said:", command)
        return command.lower()
    except:
        return ""

#Thinking Method
def think(prompt):
    try:
        full_prompt = f"""You are Smarty, a smart and informative voice assistant. Keep
responses brief and conversational , do not include emojis (3-5 sentences max).
User: {prompt}
```

```
Smarty:"""
        response = requests.post(
            OLLAMA_URL,
            json={
                "model": OLLAMA_MODEL,
                "prompt": full_prompt,
                "stream": False
            },
            timeout=25
        )
        if response.status_code == 200:
            result = response.json()
            return result.get("response", "I'm thinking...").strip()
        else:
            return "I'm having trouble processing that request."
    except requests.exceptions.ConnectionError:
        return "I can't connect to my AI brain. Is Ollama running?"
    except Exception as e:
        print(f"Error in think(): {e}")
        return "Something went wrong with my thinking process."

#Apps Management
def close_app(app_name):
    for proc in psutil.process_iter(['name']):
        try:
            if app_name.lower() in proc.info['name'].lower():
                proc.kill()
                return True
        except (psutil.NoSuchProcess, psutil.AccessDenied):
            pass
    return False

def play_on_spotify(query):
    speak(f"Playing {query} on Spotify.")
```

```
    search_url = f"https://open.spotify.com/search/{query}"
    webbrowser.open(search_url)

#START
time.sleep(1)
speak("Smarty AI system online, Hello Sir!.")

#MAIN LOOP
while True:
    command = listen()
    if not command:
        continue

    # BASIC TALKBACK & GREETINGS
    if command in ["hi", "hello", "hey smarty","smarty"]:
        speak("Hello Sir! How can I make your day easier?")
        continue

    # EXIT / STOP
    if any(word in command for word in ["exit", "stop", "shutdown", "shut down","off",
"bye"]):
        speak("Goodbye Sir. Smarty shutting down.")
        time.sleep(3)
        break
```

C. SAMPLE INPUT

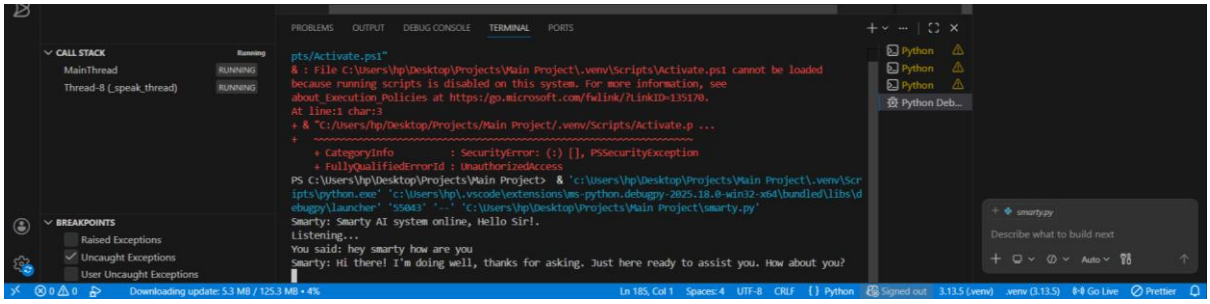


Fig 6 : Sample Talkback response checking for Smarty AI voice assistant.

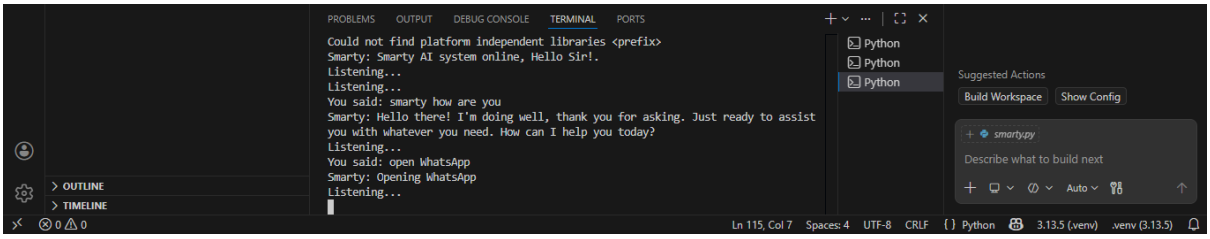


Fig 7 : To Open Applications using Voice Commands .

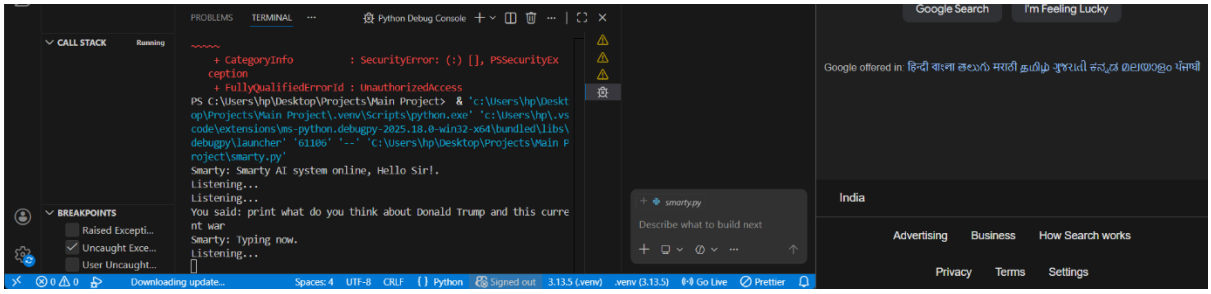


Fig 8 :Checking Auto Typing functions using Voice in Google Browser .

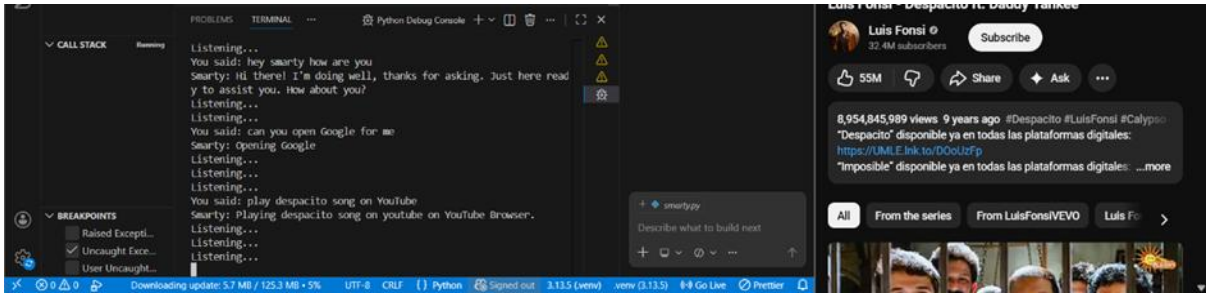


Fig 9 : To Play Media using Voice Commands .

D. SAMPLE OUTPUT

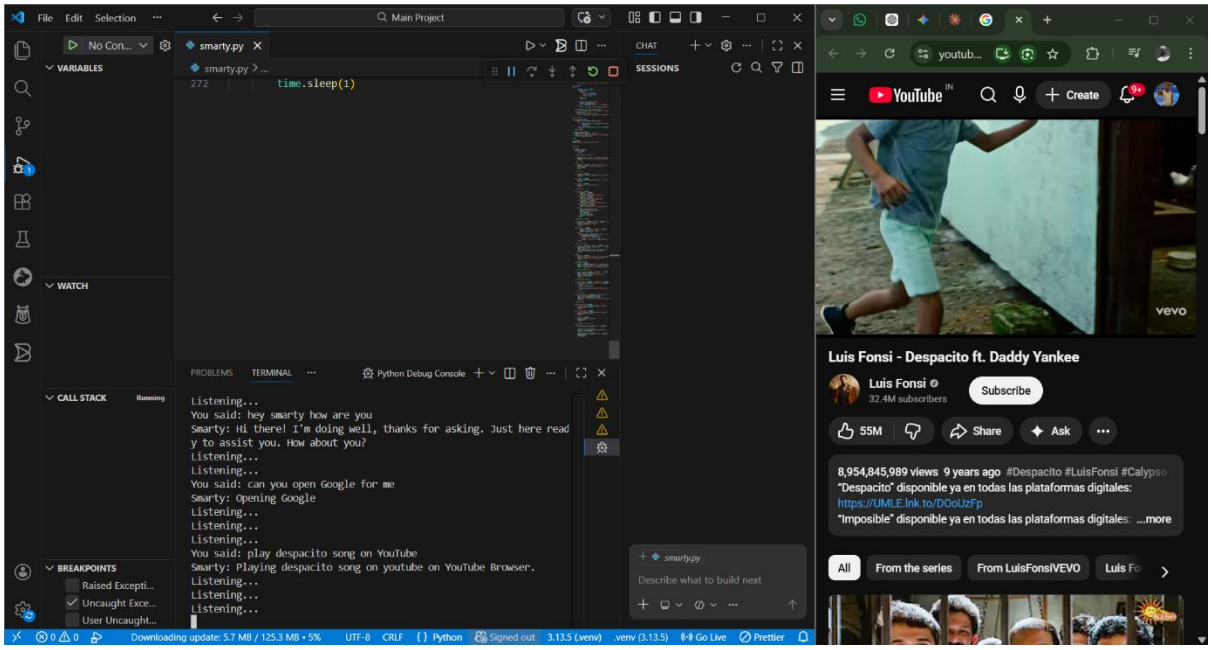


Fig 10 : Playing Videos on YouTube Web Browser on Command

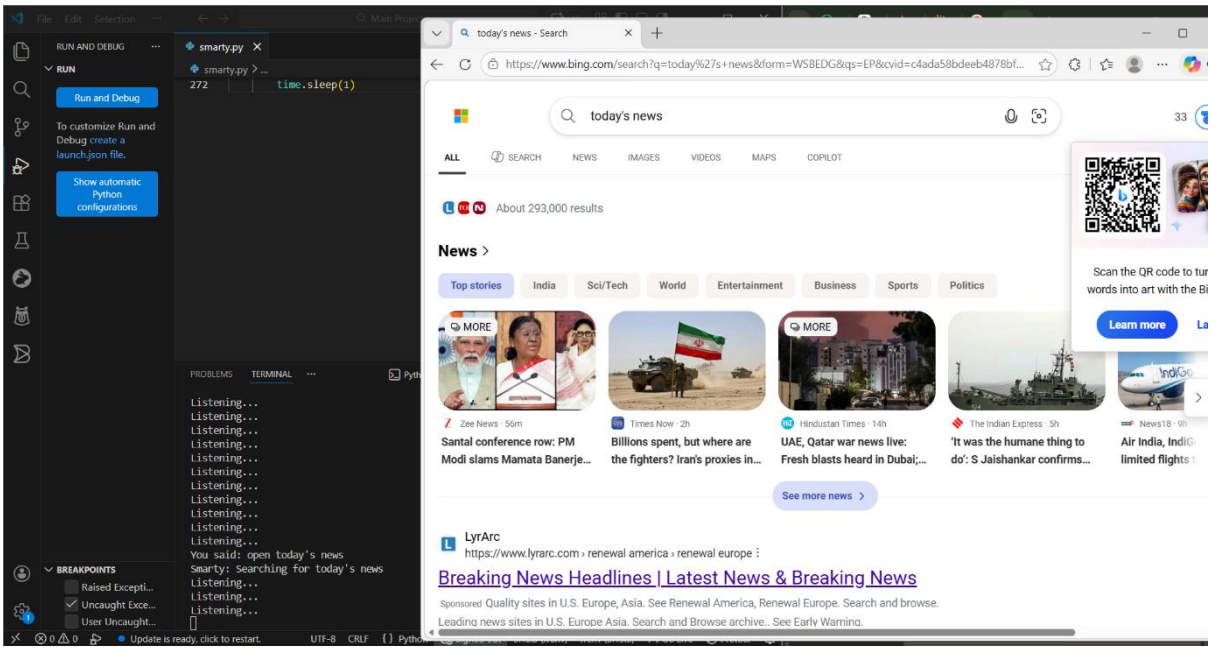


Fig 11: Displays Today's news on Command.

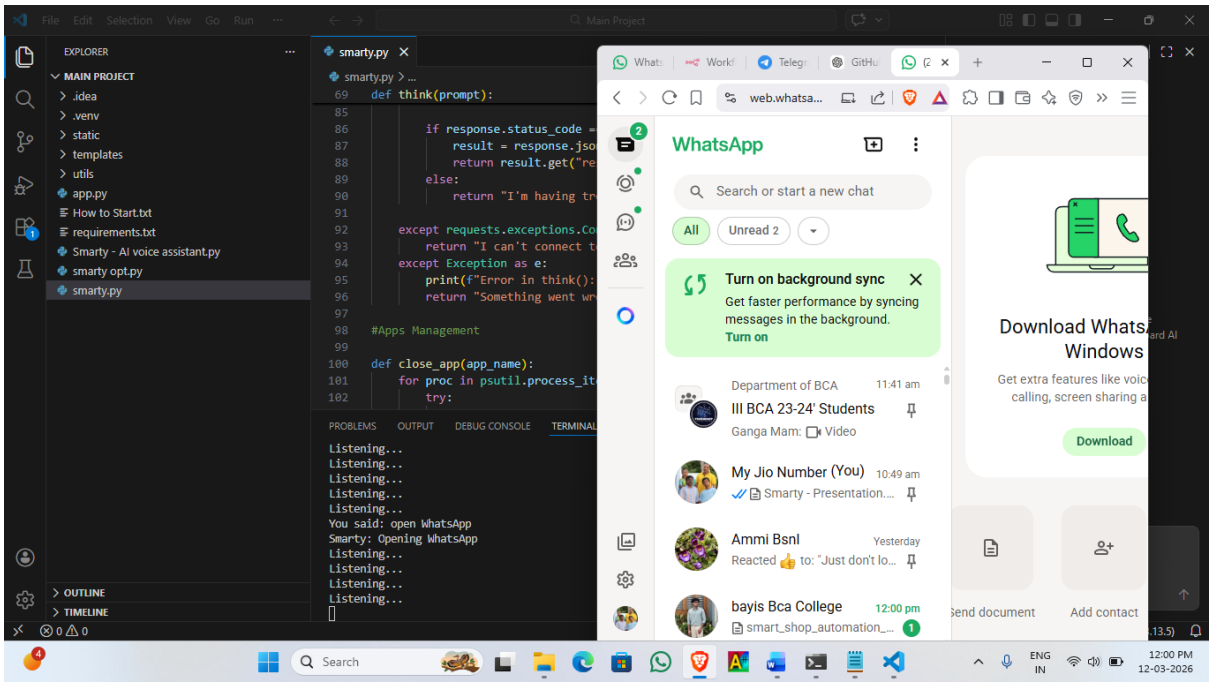


Fig 12 : Opens WhatsApp Web on Command

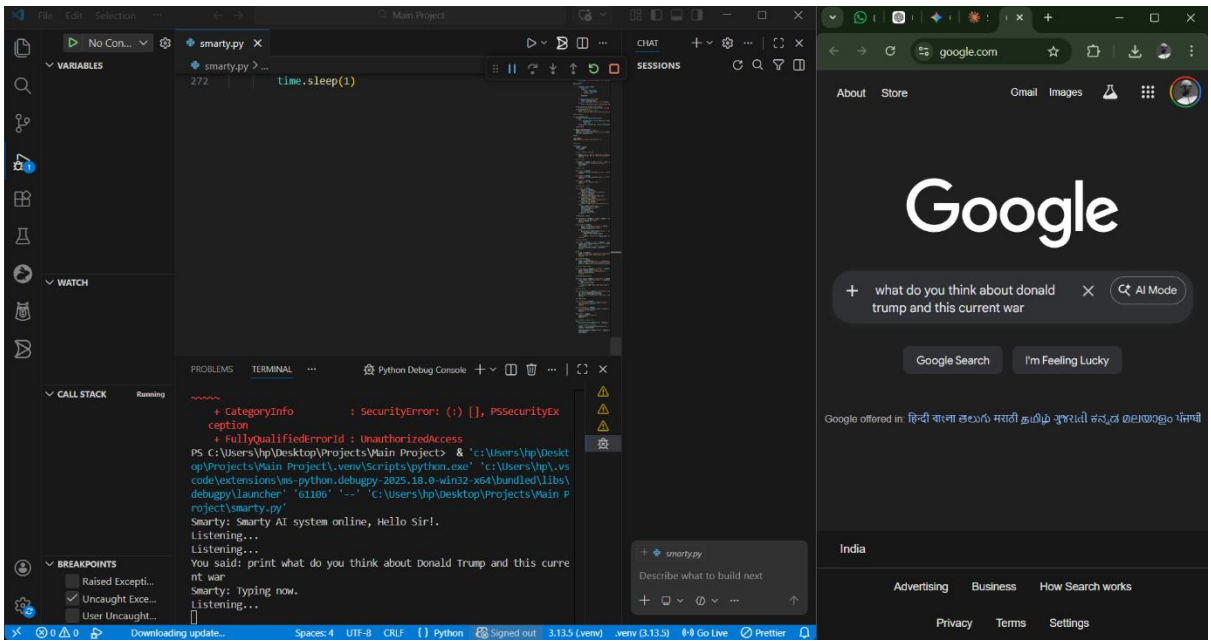


Fig 13 : Auto Typing in Google Browser